



INSTITUTO FEDERAL DA PARAÍBA
CAMPUS CAMPINA GRANDE
BACHARELADO EM ENGENHARIA DA COMPUTAÇÃO
DISCIPLINA DE ESTRUTURAS DE DADOS
PROF. VICTOR ANDRÉ PINHO DE OLIVEIRA

Estruturas de Dados

Aula 6 - ED - Pilhas

Introdução

Olá,

No assunto anterior, nós concluímos a implementação das Listas. Implementamos uma Lista Linear Sequencial, uma Lista Linear Simplesmente Encadeada e uma Lista Linear Duplamente Encadeada. Todas de forma não Ordenada e Ordenada.

Hoje veremos uma Estrutura de Dados completamente nova: a **Pilha**. Usando as habilidades já adquiridas, veremos que podemos implementar uma Pilha usando como base uma Lista Sequencial ou uma Lista Simplesmente Encadeada.

Prometo que você irá achar super simples...

Pilhas

Uma **Pilha**, à semelhança de uma Lista, é uma estrutura que armazena elementos do mesmo tipo. A diferença, contudo, está na política de acesso aos dados, em que o último elemento inserido deve ser o primeiro a ser retirado. Exatamente por isso, a Estrutura Pilha é conhecida pelo termo LIFO (*Last In First Out* - Último a Entrar é o Primeiro a Sair). Desse modo, podemos enxergar uma Pilha como uma Lista que sempre insere um novo elemento no fim e sempre obtém um elemento do fim.

Como uma Pilha é uma variante de uma Lista, apenas com uma política que restringe a forma como os dados podem ser obtidos/acessados, podemos implementá-la usando como base uma Lista Sequencial ou uma Lista Simplesmente Encadeada. Nesta aula, veremos essas duas formas de implementação.

Pilha com base em Lista Sequencial

Vamos começar implementando uma Pilha usando como base uma Lista Sequencial.

Algumas funções serão aproveitadas, outras foram levemente modificadas e outras foram descartadas. Ao invés da função inserir, agora temos a função **push**, que insere sempre os elementos no fim da Lista. Ao invés da função obter, temos a função **pop**

que retorna o último elemento da Lista já removendo-o. Em outras palavras, push empilha e pop desempilha!

Código

Segue abaixo o código.

```
#include <stdio.h>
#include <stdlib.h>

const unsigned MAX = 10;
int l[MAX], pos = 0;

void push(int elemento);
int pop();

int tamanho();
void imprimir();
void apagar();

int main(void) {
    return 0;
}

void push(int elemento){
    if (pos < MAX)
        l[pos++] = elemento;
    else
        printf("Não foi possível inserir %d. Pilha
cheia.\n",elemento);
}

int pop(){
    if (pos)
        return l[--pos];
    else {
        printf("Não há elementos. Pilha vazia.\n");
        exit(1);
    }
}

int tamanho(){
    return pos;
}
```

```

}

void imprimir(){
    printf("P[ ");
    for (int i = pos -1 ; i >= 0 ; i--)
        printf("%d ",l[i]);
    printf("]\n");
}

void apagar(){
    pos = 0;
}

```

O código acima mostra uma forma de implementar uma Pilha baseada em Lista Sequencial. É preciso de:

- algumas variáveis e uma constante:
 - uma constante **MAX**, para definir o tamanho máximo da Pilha
 - um array **l**, para armazenar os elementos da Pilha
 - uma variável **pos**¹, para controlar o acesso (push e pop) aos elementos da Pilha.
- duas funções base:
 - **push**, para empilhar um novo elemento na Pilha
 - **pop**, para desempilhar o último elemento da Pilha
- algumas funções auxiliares:
 - **tamanho**, para obter o tamanho da Pilha
 - **imprimir**, para exibir toda a Pilha
 - **apagar**, para zerar a Pilha

Como estamos implementando uma Pilha de inteiros, tanto o array **l** quanto as funções foram implementados esperando receber tal tipo.

Adicionalmente, à semelhança do que temos feito, você pode declarar todas as suas funções (assinaturas) acima da main e deixar para implementá-las em qualquer outro lugar abaixo no arquivo.

Lembre-se que, por estarmos implementando uma Pilha, que é um tipo de Estrutura de Dados, não podemos manipular o array diretamente, mas somente por meio das funções. As funções ditam as regras de acesso à Estrutura. Assim, basta entendermos como cada função funciona.

Funções base

Começaremos pelas funções base da Estrutura.

¹ Algumas implementações trazem o nome topo ao invés de pos. Contudo, resolvi manter o nome da variável como pos para que o aluno notasse as semelhanças com o código da Lista Sequencial.

Função push

A função push tem o objetivo de inserir um elemento na Pilha.

```
void push(int elemento){
    if (pos < MAX)
        l[pos++] = elemento;
    else
        printf("Não foi possível inserir %d. Pilha
cheia.\n", elemento);
}
```

A função push não retorna nada e recebe o elemento a ser inserido. Primeiramente, verificamos se ainda há espaço para novos elementos. Em caso afirmativo, inserimos na posição pos e incrementamos a variável. Note que foi usado o pós-incremento, o que quer dizer que atribuímos o elemento na posição pos do array l. Só depois é que o incremento é, de fato, aplicado.

Em caso negativo, simplesmente avisamos que não é possível inserir o elemento na Pilha por ela já estar cheia.

Note que, se olharmos para a Pilha como uma Lista, a função push simplesmente insere o elemento no fim da Lista.

Função pop

A função pop tem o objetivo de retornar o último elemento da Pilha.

```
int pop(){
    if (pos)
        return l[--pos];
    else {
        printf("Não há elementos. Pilha vazia.\n");
        exit(1);
    }
}
```

A função retorna o elemento no topo da Pilha (nesse caso é do tipo inteiro) e não recebe nada. Primeiro verificamos se a Pilha não está vazia, avaliando a variável pos. Se pos for 0 (zero) é porque não há nenhum elemento na Pilha. Como pos vai conter sempre um índice à frente do último elemento útil do array, decrementamos pos antes de acessar o elemento e já retornamos. Note que a função pop retorna o elemento removendo-o da Pilha (na verdade ele ainda vai estar no array, mas inacessível).

Caso a Pilha esteja vazia, imprimimos o aviso na tela e encerramos o programa. Como a função deve retornar algum elemento, não há como continuar se não houver elemento.

Mais uma vez, se olharmos para a Pilha como uma Lista, veremos que a função pop simplesmente retorna e remove o elemento do fim da Lista.

Funções auxiliares

E agora, veremos as funções auxiliares.

Função tamanho

Retorna o tamanho da Pilha.

```
int tamanho(){  
    return pos;  
}
```

Basicamente, a função retorna pos, pois a variável vai sempre coincidir com a quantidade de elementos.

Função imprimir

A função imprimir não recebe nem retorna nada. Apenas imprime o conteúdo da Pilha na tela.

```
void imprimir(){  
    printf("P[ ");  
    for (int i = pos - 1 ; i >= 0 ; i--)  
        printf("%d ", l[i]);  
    printf("]\n");  
}
```

Muito semelhante à função imprimir das Listas Sequenciais. No entanto, resolvi apresentar os dados de trás para frente para manter consistência com a função imprimir da implementação de Pilha com base em Lista Simplesmente Encadeada. Note que, na exibição, acrescentei um P antes dos colchetes para indicar que temos uma Pilha.

Função apagar

A função apagar não recebe nem retorna nada. Tem o objetivo de zerar a Pilha.

```
void apagar(){  
    pos = 0;  
}
```

A função está exatamente igual à implementação das Listas Sequenciais. Ao atribuir 0 (zero) à pos estamos dizendo que “não temos elementos na Pilha”. Na verdade, como bem sabemos, os elementos estarão no array, mas inacessíveis através da Estrutura. Assim, para todos os efeitos, é como se todos os elementos fossem apagados/removidos da Pilha.

Pilha com base em Lista Simplesmente Encadeada

Continuando, agora vou mostrar como implementar uma Pilha usando como base uma Lista Simplesmente Encadeada.

De semelhante modo, algumas funções serão aproveitadas, outras foram levemente modificadas e outras foram descartadas. Ao invés da função `inserir`, agora temos a função **push**, que insere sempre os elementos no fim da Lista. Ao invés da função `obter`, temos a função **pop** que retorna o último elemento da Lista já removendo-o.

Código

Segue abaixo o código.

```
#include <stdio.h>
#include <stdlib.h>

struct sNODE{
    int dado;
    struct sNODE *prox;
};

struct sNODE *topo = NULL;

void push(int dado);
int pop();

int tamanho();
void imprimir();
void apagar();

int main(void) {
    return 0;
}

void push(int dado){
    struct sNODE *novo = (struct sNODE*) malloc(sizeof(struct sNODE));
    novo->dado = dado;
    novo->prox = NULL;

    if (!topo)
```

```

        topo = novo;
    else{
        novo->prox = topo;
        topo = novo;
    }
}

int pop(){
    if (topo) {
        int dado = topo->dado;
        struct sNODE *tmp = topo;

        topo = topo->prox;

        free(tmp);

        return dado;
    }
    else {
        printf("Não há elementos. Pilha vazia.\n");
        exit(1);
    }
}

int tamanho(){
    struct sNODE *aux = topo;
    int tam = 0;

    while (aux){
        tam++;
        aux = aux->prox;
    }

    return tam;
}

void imprimir(){
    struct sNODE *aux = topo;

    printf("P[ ");
    while (aux){

```

```

        printf("%d ", aux->dado);
        aux = aux->prox;
    }
    printf("]\n");
}

void apagar(){
    struct sNODE *aux = topo, *ant;

    while (aux){
        ant = aux;
        aux = aux->prox;
        free(aux);
    }
    topo = NULL;
}

```

O código acima mostra uma forma de implementar uma Pilha baseada em Lista Simplesmente Encadeada. É preciso de:

- algumas variáveis e um registro:
 - um registro **sNODE**, contendo um campo **dado** que armazena o elemento e um campo **prox** que aponta para o próximo nó
 - um ponteiro **topo**, que vai sempre apontar para o último nó. Como em uma Pilha a manipulação sempre envolve o último elemento, não precisamos de um ponteiro para o primeiro nó
- duas funções base:
 - **push**, para empilhar um novo elemento na Pilha
 - **pop**, para desempilhar o último elemento da Pilha
- algumas funções auxiliares:
 - **tamanho**, para obter o tamanho da Pilha
 - **imprimir**, para exibir toda a Pilha
 - **apagar**, para zerar a Pilha

Como estamos implementando uma Pilha de inteiros, tanto o campo **dado** do registro **sNODE** quanto as funções foram implementados esperando receber tal tipo.

Os demais comentários da seção anterior continuam válidos aqui.

Funções base

Começaremos pelas funções base da Estrutura.

Função push

A função **push** tem o objetivo de inserir um elemento na Pilha.

```

void push(int dado){
    struct sNODE *novo = (struct sNODE*) malloc(sizeof(struct
sNODE));
    novo->dado = dado;
    novo->prox = NULL;

    if (!topo)
        topo = novo;
    else{
        novo->prox = topo;
        topo = novo;
    }
}

```

A função push não retorna nada e recebe o elemento a ser inserido. Primeiramente alocamos um novo nó e atribuímos o parâmetro dado ao campo dado e NULL ao campo prox. Como o elemento a ser inserido vai ser colocado sempre no final, mais à frente realizaremos o ajuste do campo prox.

Em seguida, verificamos o ponteiro topo para identificar se a Pilha contém ou não elementos. Caso topo seja NULL, o novo nó é o único elemento da Pilha, então prox deve continuar NULL.

Por outro lado, se já houver elementos na Pilha, fazemos com que o novo nó aponte para o topo anterior. Só depois, atualizamos o topo para o novo nó.

Note que aqui estamos utilizando o raciocínio de uma Lista invertida, se comparado à maneira como implementamos uma Lista Simplesmente Encadeada. Fizemos assim porque só temos um ponteiro para o topo da Pilha. Assim, o nó cujo campo prox seja NULL vai indicar que ele é o outro extremo da Lista (a base da Pilha), e, consequentemente, o último nó da Pilha a ser obtido.

Função pop

A função pop tem o objetivo de retornar o último elemento da Pilha.

```

int pop(){
    if (topo) {
        int dado = topo->dado;
        struct sNODE *tmp = topo;

        topo = topo->prox;

        free(tmp);

        return dado;
    }
}

```

```

    }
    else {
        printf("Não há elementos. Pilha vazia.\n");
        exit(1);
    }
}

```

A função retorna o elemento no topo da Pilha (nesse caso é do tipo inteiro) e não recebe nada. Primeiro avaliamos o ponteiro topo para verificar se a Pilha não está vazia. Se a Pilha não estiver vazia, acessamos o nó topo e salvamos o dado a ser retornado. Em seguida, guardamos o ponteiro para topo em tmp e fazemos topo avançar um nó. Já que estamos dando um pop, temos que remover o nó que guardava o elemento: free(temp). Depois podemos retornar o dado.

Caso a função seja invocada e a Pilha esteja vazia, como não há dado algum para retornar, a função exibe que não há elementos e encerra a aplicação com exit(1) - saída de erro.

Funções auxiliares

E agora, veremos as funções auxiliares.

Função tamanho

Retorna o tamanho da Pilha.

```

int tamanho(){
    struct sNODE *aux = topo;
    int tam = 0;

    while (aux){
        tam++;
        aux = aux->prox;
    }

    return tam;
}

```

Basicamente, a função é exatamente igual à implementação na Lista Simplesmente Encadeada. A diferença é que aux recebe topo ao invés de ini. A função faz com que o ponteiro aux percorra toda a lista incrementando uma variável tam. Ao final, retorna tam.

Função imprimir

A função imprimir não recebe nem retorna nada. Apenas imprime o conteúdo da Pilha na tela.

```

void imprimir(){
    struct sNODE *aux = topo;

    printf("P[ ");
    while (aux){
        printf("%d ", aux->dado);
        aux = aux->prox;
    }
    printf("]\n");
}

```

Outra função muito semelhante à função imprimir da Lista Simplesmente Encadeada. No entanto, como temos apenas um ponteiro para o topo, ao imprimir a partir dele, os dados são exibidos de trás para frente. Também usamos o P antes dos colchetes na exibição para diferenciar a Pilha de uma Lista.

Função apagar

A função apagar não recebe nem retorna nada. Tem o objetivo de zerar a Pilha.

```

void apagar(){
    struct sNODE *aux = topo, *ant = NULL;

    while (aux){
        ant = aux;
        aux = aux->prox;
        free(ant);
    }
    topo = NULL;
}

```

Mais uma função semelhante à implementação da Lista Simplesmente Encadeada. Contudo, ao invés de aux receber ini, mais uma vez atribuímos topo. A ideia da função é correr aux pelos nós mantendo sempre um ponteiro ant apontando para o nó imediatamente anterior a aux. Na medida que aux avança, a região apontada por ant é liberada com free(ant).

Por fim, observe que, do ponto de vista de quem usa a Pilha, praticamente não existem diferenças. A única que poderíamos apontar é o limite máximo de elementos imposto pela implementação com Lista Sequencial, o que poderia ser facilmente resolvido se ao invés de uma array estático tivéssemos um ponteiro para um array alocado dinamicamente, realocando o tamanho do array em caso de necessidade. Outra opção

seria impor um tamanho máximo na quantidade de elementos na implementação de Pilha com Lista Simplesmente Encadeada. São muitas opções, o céu é o limite.

Na aula de hoje implementamos a Estrutura de Dados Pilha. Se você entendeu direitinho a implementação das Listas, tenho absoluta certeza que tirou a aula de hoje de letra.

Na próxima aula iremos tratar sobre a Estrutura de Dados chamada de Fila. De semelhante modo, veremos como implementar uma Fila com Listas Sequenciais e com Listas Encadeadas.

Bons estudos e até a próxima!